



UNIVERSITÀ DI PARMA

DIPARTIMENTO DI SCIENZE MATEMATICHE, FISICHE E INFORMATICHE

Corso di Laurea Triennale in Informatica

Metodologie per l'Orientamento al Pensiero Computazionale: un Caso di Studio basato sull'Algoritmo Heap Sort

*Methodologies for Computational Thinking Orientation: A Case
Study based on the Heap Sort Algorithm*

CANDIDATO:

Kenneth James Nna Minkousse

RELATORE:

Prof. Vincenzo Bonnici

Indice

Introduzione	1
1 Orientamento al Pensiero Computazionale	3
1.1 Cos'è il pensiero computazionale	3
1.1.1 I quattro pilastri del Pensiero Computazionale	3
1.2 Perché insegnarlo: obiettivi educativi e formativi	4
1.3 Metodologie didattiche per l'orientamento	4
1.3.1 Programmazione visuale e attività unplugged	5
1.3.2 Learning-by-doing e problem solving	5
1.3.3 Gamification e Ambienti Interattivi	5
1.3.4 Game-based learning (GBL)	5
1.4 L'algoritmo Heap Sort come strumento didattico	6
1.4.1 Fondamenti teorici dell'Heap	6
1.4.2 Analisi dell'Algoritmo	6
1.4.3 Analisi della Complessità (Big O Notation)	7
2 Descrizione e Analisi del Progetto	9
2.1 Obiettivi del Progetto di Tirocinio	9
2.1.1 1. Obiettivi Didattici e Cognitivi	9
2.1.2 2. Obiettivi Tecnici e Architettureali	10
2.1.3 3. Obiettivi di User Experience (Gamification)	10
2.2 Descrizione Dettagliata dell'Applicazione	11
2.2.1 Interfaccia Utente (UI) e Layout	11
2.2.2 Flusso di Interazione (User Journey)	11
2.3 Strumenti e Tecnologie: Analisi Tecnica	12
2.3.1 HTML5: Semantica e Accessibilità	12
2.3.2 CSS3 Avanzato: Grid, Flexbox e Variabili	12
2.3.3 JavaScript (ES6+) Moderno	13
2.4 Analisi Approfondita dei Requisiti	13
2.4.1 Requisiti Funzionali (RF)	13
2.4.2 Requisiti Non Funzionali (RNF)	14

Introduzione

L'informatica e il **pensiero computazionale** hanno assunto un ruolo centrale nella formazione moderna, diventando competenze imprescindibili per la comprensione della realtà digitale che ci circonda. Tuttavia, l'apprendimento delle strutture dati e degli algoritmi fondamentali rappresenta spesso uno scoglio significativo per gli studenti universitari e delle scuole superiori. Concetti astratti come la ricorsione, la complessità asintotica o la manipolazione di alberi binari, se affrontati esclusivamente attraverso lo studio teorico sui libri di testo o tramite diagrammi statici, possono risultare ostici, aridi e privi di un riscontro concreto immediato.

Un esempio emblematico di questa difficoltà è rappresentato dall'algoritmo di ordinamento **Heap Sort**. Sebbene sia uno degli algoritmi più eleganti ed efficienti (con una complessità temporale di $O(n \log n)$), la sua comprensione richiede uno sforzo cognitivo notevole: lo studente deve visualizzare mentalmente una struttura dati (l'array) che viene interpretata logica come un'altra (un albero binario quasi completo), mantenendo costantemente traccia delle relazioni matematiche tra gli indici (padre, figlio sinistro, figlio destro). Questa "doppia natura" dell'Heap crea spesso confusione e ostacola l'interiorizzazione profonda del meccanismo di ordinamento.

Questo divario tra teoria astratta e comprensione pratica è il punto di partenza del presente progetto di tirocinio. L'obiettivo primario è stato quello di progettare e sviluppare un'applicazione web interattiva, denominata "**Heap Sort Challenge**", che funga da ponte tra la definizione formale dell'algoritmo e la sua esecuzione dinamica. Sfruttando le moderne tecnologie web (HTML5, CSS3, JavaScript ES6) e adottando un approccio pedagogico basato sulla **Gamification**, il progetto trasforma il processo di apprendimento da un'attività passiva di studio a una sfida attiva e coinvolgente.

A differenza delle tradizionali animazioni algoritmiche (presenti in molti siti didattici), dove l'utente si limita a premere "Play" e osservare le barre muoversi, "Heap Sort Challenge" pone lo studente al centro dell'azione. L'applicazione implementa una modalità "Uomo contro Macchina": l'utente deve competere contro una CPU simulata, prendendo decisioni in tempo rea-

le su quali nodi scambiare per soddisfare le proprietà matematiche del Max-Heap. Questo approccio "*Learning-by-Doing*" costringe l'utente a ragionare come l'algoritmo, valutando le condizioni e prevedendo le conseguenze di ogni scambio, favorendo così un apprendimento più duraturo e profondo.

Dal punto di vista tecnico, il progetto è stato l'occasione per applicare metodologie di ingegneria del software rigorose in un contesto frontend. L'applicazione è stata architettata seguendo il pattern **Model-View-Presenter (MVP)**, garantendo una netta separazione tra la logica algoritmica (il Modello), la rappresentazione visiva nel DOM (la Vista) e la logica di controllo e interazione (il Presenter). Questa scelta non solo ha facilitato lo sviluppo e il testing, ma rappresenta di per sé un caso di studio interessante su come strutturare applicazioni web client-side moderne e manutenibili senza l'ausilio di framework pesanti.

Nel seguito di questo elaborato, il lavoro svolto verrà presentato secondo la seguente strutturazione:

Nel **Capitolo 1** verrà introdotto il contesto teorico del Pensiero Computazionale, analizzando come gli strumenti interattivi possano potenziare le capacità di astrazione, decomposizione e riconoscimento di pattern, competenze chiave per ogni informatico.

Il **Capitolo 2** sarà dedicato all'analisi approfondita dell'algoritmo Heap Sort. Verranno richiamati i fondamenti teorici, le proprietà della struttura dati Heap e la logica matematica che ne governa il funzionamento, fornendo il background necessario per comprendere le scelte implementative.

Nel **Capitolo 3** si affronterà la fase di Progettazione e Design. Verranno illustrati i diagrammi UML che descrivono l'architettura del sistema, le scelte di User Interface (UI) e User Experience (UX) mirate a massimizzare l'usabilità, e la struttura del pattern MVP adottato.

Il **Capitolo 4** entrerà nel dettaglio dell'Implementazione tecnica. Verranno discussi gli aspetti più rilevanti del codice sorgente, come la gestione del ciclo di vita del gioco, l'implementazione della "Intelligenza Artificiale" avversaria e le tecniche CSS utilizzate per garantire animazioni fluide e performanti.

Infine, nel **Capitolo 5**, verranno trattate le Conclusioni, analizzando i risultati ottenuti rispetto agli obiettivi prefissati, discutendo le limitazioni attuali del sistema e proponendo possibili sviluppi futuri per estendere la piattaforma ad altri algoritmi o modalità di gioco.

Capitolo 1

Orientamento al Pensiero Computazionale

1.1 Cos'è il pensiero computazionale

Il pensiero computazionale, termine reso celebre da Jeannette Wing nel 2006, rappresenta "un approccio fondamentale per la risoluzione di problemi, la progettazione di sistemi e la comprensione del comportamento umano che attinge ai concetti fondamentali dell'informatica". Non si tratta semplicemente di "pensare come un computer", che è un mero esecutore di istruzioni, ma piuttosto di pensare con i livelli di astrazione necessari per rendere un problema risolvibile da una macchina. È un processo cognitivo che coinvolge diverse dimensioni chiave:

1.1.1 I quattro pilastri del Pensiero Computazionale

1. **Astrazione:** È la capacità di filtrare i dettagli superflui per concentrarsi sugli aspetti essenziali di un problema. *Esempio nel progetto:* Nel gioco Heap Sort, l'astrazione è rappresentata dal modo in cui i numeri vengono visualizzati come nodi di un albero. I dettagli di basso livello (come i bit che compongono il numero in memoria) sono nascosti; ciò che conta è la relazione logica padre-figlio.
2. **Decomposizione:** Consiste nel suddividere un problema complesso in sotto-problemi più piccoli e gestibili. *Esempio nel progetto:* L'ordinamento di un intero array viene decomposto in due fasi principali (Heapify e Sorting), e a sua volta ogni fase è composta da singole operazioni atomiche di "swap" e "confronto". Lo studente impara che risolvere un problema grande significa risolverne tanti piccoli in sequenza.

3. **Riconoscimento di pattern:** La capacità di identificare somiglianze, tendenze e regolarità nei dati. *Esempio nel progetto:* Riconoscere che ogni sotto-albero deve soddisfare la stessa proprietà (es. Max-Heap) permette di applicare la stessa logica ricorsivamente a tutta la struttura.
4. **Algoritmica:** La definizione di una serie di passi ordinati e precisi (un algoritmo) per raggiungere un obiettivo. Non basta "sapere" la soluzione, bisogna saperla descrivere in modo che sia replicabile.

1.2 Perché insegnarlo: obiettivi educativi e formativi

L'introduzione del pensiero computazionale nelle scuole risponde alla necessità di formare cittadini consapevoli in una società digitale (Digital Literacy). Tuttavia, il valore educativo trascende l'alfabetizzazione informatica:

- **Meta-cognizione:** Imparare a programmare o a simulare algoritmi aiuta gli studenti a riflettere sul proprio modo di pensare ("thinking about thinking").
- **Problem Solving Trasversale:** Le tecniche apprese (decomposizione, astrazione) sono applicabili in matematica, scienze, linguistica e persino nella gestione di progetti quotidiani.
- **Gestione dell'Errore:** In informatica, l'errore (bug) non è un fallimento, ma un'informazione utile per correggere il processo. Questo approccio favorisce una "Growth Mindset" (mentalità di crescita), riducendo l'ansia da prestazione scolastica.
- **Creatività Computazionale:** Fornire gli strumenti per passare da consumatori passivi di tecnologia (fruitori di app) a creatori attivi, capaci di esprimere le proprie idee attraverso il codice o la logica algoritmica.

1.3 Metodologie didattiche per l'orientamento

Per insegnare efficacemente questi concetti, è necessario allontanarsi dalla lezione frontale trasmissiva ("Instructionism") per abbracciare metodologie costruttiviste ("Constructionism" di Seymour Papert).

1.3.1 Programmazione visuale e attività unplugged

La programmazione visuale (es. Scratch, Blockly) riduce il carico cognitivo legato alla sintassi del codice (punti e virgola, parentesi), permettendo di concentrarsi sulla semantica e sulla logica. Le attività *CS Unplugged* (senza computer) utilizzano giochi fisici, carte, corda o movimenti corporei per introdurre concetti informatici. Ad esempio, ordinare una fila di studenti per altezza è un modo fisico per spiegare un algoritmo di sorting. Queste attività rendono l'informatica accessibile anche in contesti con scarse risorse tecnologiche.

1.3.2 Learning-by-doing e problem solving

L'approccio "imparare facendo" è fondamentale. Invece di memorizzare definizioni astratte di "ciclo" o "variabile", gli studenti affrontano problemi reali. La conoscenza emerge come sottoprodotto dell'attività di risoluzione del problema. Nel contesto del nostro progetto, lo studente non "studia" l'Heap Sort, ma lo "esegue", internalizzando le regole attraverso la pratica.

1.3.3 Gamification e Ambienti Interattivi

La *gamification* applica meccaniche di gioco (punti, livelli, classifiche, sfide a tempo) a contesti non ludici per aumentare la motivazione intrinseca ed estrinseca. Gli ambienti interattivi offrono un **feedback immediato**. Nel modello tradizionale (compito in classe), il feedback (voto) arriva giorni dopo l'esecuzione, quando ormai il momento di apprendimento è passato. In un software educativo, se lo studente sbaglia un passaggio, il sistema lo notifica istantaneamente (es. nodo rosso), permettendo una correzione in tempo reale. Questo ciclo di feedback rapido è essenziale per l'apprendimento profondo.

1.3.4 Game-based learning (GBL)

A differenza della gamification che "aggiunge" elementi di gioco a un processo esistente, il *Game-Based Learning* utilizza veri e propri giochi come veicolo principale per l'apprendimento. Il gioco diventa il "libro di testo" interattivo. In "Heap Sort Challenge", il contenuto didattico (l'algoritmo) è intrinseco alle meccaniche di gioco: per vincere, devi necessariamente comprendere l'algoritmo.

1.4 L'algoritmo Heap Sort come strumento didattico

L'Heap Sort è stato scelto come oggetto di questo progetto per la sua ricchezza concettuale. Non è il più semplice (come il Bubble Sort) né sempre il più veloce nella pratica (come il QuickSort), ma offre spunti didattici unici sulla gestione efficiente dei dati.

1.4.1 Fondamenti teorici dell'Heap

Un **Binary Heap** è una struttura dati che può essere vista come un albero binario quasi completo. Un albero si dice "quasi completo" se tutti i livelli sono riempiti completamente, tranne eventualmente l'ultimo, che viene riempito da sinistra a destra. Questa proprietà strutturale è cruciale perché permette di memorizzare l'albero in un array compatto senza l'uso di puntatori espliciti, garantendo un'efficienza spaziale ottimale.

Dato un nodo all'indice i (0-indexed):

- Il figlio sinistro è a: $2i + 1$
- Il figlio destro è a: $2i + 2$
- Il genitore è a: $\lfloor \frac{i-1}{2} \rfloor$

Oltre alla proprietà strutturale, l'Heap deve soddisfare la *Proprietà di Ordine*:

- **Max-Heap**: Ogni nodo è maggiore o uguale ai suoi figli ($A[\text{parent}(i)] \geq A[i]$). Di conseguenza, l'elemento massimo è sempre nella radice.
- **Min-Heap**: Ogni nodo è minore o uguale ai suoi figli ($A[\text{parent}(i)] \leq A[i]$). L'elemento minimo è nella radice.

1.4.2 Analisi dell'Algoritmo

L'algoritmo Heap Sort procede in due fasi distinte, entrambe visualizzate nel software:

Fase 1: Build-Max-Heap (Costruzione)

Si parte da un array disordinato. Si interpretano i dati come un albero binario. Partendo dall'ultimo nodo non-foglia (indice $n/2 - 1$) e risalendo fino alla radice (indice 0), si applica la procedura **Heapify** verso il basso.

Questa fase trasforma l'array caotico in un Max-Heap valido. La complessità di questa fase, con un'analisi attenta, risulta essere lineare: $O(n)$.

Fase 2: Sorting (Ordinamento)

Una volta costruito l'Heap:

1. Si scambia la radice (massimo attuale) con l'ultimo elemento dell'heap ($A[n - 1]$).
2. Si riduce la dimensione considerata dell'heap di 1 (l'elemento massimo è ora "congelato" nella sua posizione finale corretta).
3. La nuova radice probabilmente viola la proprietà di Max-Heap, quindi si esegue **Heapify** sulla radice per far "affondare" il valore nella posizione corretta.
4. Si ripete fino a quando l'heap ha dimensione 1.

1.4.3 Analisi della Complessità (Big O Notation)

Uno degli obiettivi formativi avanzati è far comprendere agli studenti l'efficienza degli algoritmi.

- **Tempo:** La procedura **Heapify** su un nodo richiede tempo proporzionale all'altezza dell'albero, ovvero $O(\log n)$. Poiché nella fase di sorting eseguiamo **Heapify** n volte, la complessità totale è $O(n \log n)$. Questo vale per tutti i casi:
 - **Best Case:** $O(n \log n)$ (o $O(n)$ se le chiavi sono uguali, ma generalmente domina il logaritmo).
 - **Average Case:** $O(n \log n)$.
 - **Worst Case:** $O(n \log n)$. Questa è una proprietà molto importante rispetto al QuickSort, che nel caso peggiore degrada a $O(n^2)$.
- **Spazio:** L'Heap Sort è un algoritmo **in-place**. Richiede solo una quantità costante di memoria ausiliaria $O(1)$ per lo scambio delle variabili, a differenza del MergeSort che richiede $O(n)$ spazio extra per gli array di appoggio.

- **Stabilità:** È un algoritmo **instabile**. Lo scambio di elementi distanti può alterare l'ordine relativo di elementi con chiavi uguali. Questo è un dettaglio tecnico importante che viene spesso discusso in corsi universitari.

Algoritmo 1 Max-Heapify (Versione Ricorsiva)

Require: Array A , indice i

Ensure: Sottosalbero radicato in i conforme alla proprietà di Max-Heap

```

1:  $left \leftarrow 2i + 1$ 
2:  $right \leftarrow 2i + 2$ 
3:  $largest \leftarrow i$ 
4: if  $left < A.heapSize \wedge A[left] > A[largest]$  then
5:    $largest \leftarrow left$ 
6: if  $right < A.heapSize \wedge A[right] > A[largest]$  then
7:    $largest \leftarrow right$ 
8: if  $largest \neq i$  then
9:   Swap  $A[i]$  with  $A[largest]$ 
10:  Heapify( $A, largest$ )

```

Algoritmo 2 Heap Sort Completo

Require: Array A da ordinare

Ensure: Array A ordinato in senso crescente

```

1: Fase 1: Costruzione dell'Heap
2: for  $i \leftarrow \lfloor A.length/2 \rfloor - 1$  downto 0 do
3:   Max-Heapify( $A, i$ )
4: Fase 2: Estrazione e Ordinamento
5: for  $i \leftarrow A.length - 1$  downto 1 do
6:   Swap  $A[0]$  with  $A[i]$ 
7:    $A.heapSize \leftarrow A.heapSize - 1$ 
8:   Max-Heapify( $A, 0$ )

```

Capitolo 2

Descrizione e Analisi del Progetto

2.1 Obiettivi del Progetto di Tirocinio

Il progetto "Heap Sort Challenge" nasce dalla convergenza di due esigenze: una didattica (migliorare l'apprendimento degli algoritmi) e una tecnica (sfruttare le potenzialità del web moderno). L'obiettivo primario è decostruire la complessità dell'Heap Sort, trasformandolo da "concetto astratto da memorizzare" a "sistema dinamico da esplorare".

Gli obiettivi sono stati categorizzati secondo tre direttrici principali:

2.1.1 1. Obiettivi Didattici e Cognitivi

Basandosi sulla tassonomia di Bloom rivisitata, il software mira a supportare diversi livelli di apprendimento:

- **Ricordare e Capire (Livelli base):** Attraverso la visualizzazione, lo studente memorizza la struttura dell'albero e comprende le relazioni padre-figlio.
- **Applicare (Livello intermedio):** Nella modalità di gioco, lo studente deve applicare attivamente le regole di swap per risolvere le violazioni di heap.
- **Analizzare e Valutare (Livelli avanzati):** Confrontando la propria strategia con quella della CPU, lo studente valuta l'efficienza delle proprie mosse e analizza gli errori commessi.

Specificamente, ci si attende che alla fine dell'utilizzo lo studente sappia:

1. Identificare visivamente una violazione della proprietà di Max-Heap o Min-Heap.

2. Prevedere l'effetto di uno scambio sulla struttura globale dell'albero.
3. Spiegare perché l'estrazione avviene sempre dalla radice.

2.1.2 2. Obiettivi Tecnici e Architetture

La sfida tecnica non riguardava solo la "traduzione" dell'algoritmo in codice, ma la creazione di un sistema software robusto e manutenibile.

- **Architettura Disaccoppiata:** Implementare rigorosamente il pattern MVP (Model-View-Presenter). Il *Model* (logica dell'heap) non deve sapere nulla dell'HTML; la *View* (DOM) non deve contenere logica di business; il *Presenter* funge da direttore d'orchestra. Questo permette, ad esempio, di cambiare completamente l'interfaccia grafica senza toccare una riga dell'algoritmo.
- **Performance delle Animazioni:** Garantire i 60fps (frame per secondo) durante le animazioni di swap. Questo ha richiesto l'uso attento di proprietà CSS performanti (come 'transform' e 'opacity') evitando quelle che causano il "reflow" della pagina (come 'top', 'left', 'margin').
- **Responsive Design:** L'albero binario deve adattarsi a schermi di diverse dimensioni senza sovrapposizioni dei nodi, una sfida non banale data la natura espansiva delle strutture ad albero.

2.1.3 3. Obiettivi di User Experience (Gamification)

Per mantenere alto l'engagement ("coinvolgimento"), sono state applicate le euristiche di Nielsen e principi di Gamification:

- **Visibilità dello Stato:** L'utente deve sempre sapere di chi è il turno e qual è lo stato corrente dell'heap.
- **Feedback Immediato:** Ogni azione deve avere una reazione. Un click valido "accende" il nodo; un click invalido lo fa "tremare" (shake animation) in rosso.
- **Challenge (Sfida):** La presenza della CPU non è cosmetica. L'algoritmo avversario è stato tarato per essere veloce ma non istantaneo, creando una "race condition" psicologica che spinge l'utente a pensare più in fretta.

2.2 Descrizione Dettagliata dell'Applicazione

L'applicazione si presenta come una "Single Page Application" (SPA), dove l'intero flusso di gioco avviene senza ricaricamenti di pagina, garantendo un'esperienza fluida simile a un'app nativa.

2.2.1 Interfaccia Utente (UI) e Layout

L'interfaccia è divisa logicamente in tre macro-aree:

1. **Header e Controlli (Top Bar):** Qui risiedono i controlli globali. Un selettore (dropdown) permette di scegliere tra "Min Heap" e "Max Heap". Un pulsante prominente "Start Game" avvia la simulazione. Un display del tempo e dei turni tiene traccia della partita.
2. **Area di Gioco (Main Stage):** Lo schermo è spaccato verticalmente (Split Screen):
 - **Sinistra (Player Zone):** È l'area interattiva. L'utente vede il proprio heap. I nodi sono cerchi cliccabili, connessi da linee (SVG o bordi CSS) che rappresentano gli archi dell'albero. Sotto l'albero, viene visualizzata la rappresentazione lineare (Array View) che si aggiorna in sincronia, fondamentale per far capire la corrispondenza Albero $\leftrightarrow$ Array.
 - **Destra (CPU Zone):** È l'area speculare ma non interattiva. Qui l'utente osserva l'algoritmo eseguito dalla macchina. I nodi si muovono da soli, offrendo un esempio visivo corretto da imitare.
3. **Area di Feedback (Overlay/Toast):** Messaggi temporanei appaiono per notificare eventi critici: "Mossa non valida!", "Heap Costruito!", "Hai Vinto!".

2.2.2 Flusso di Interazione (User Journey)

Immaginiamo una sessione tipica di uno studente:

1. **Ingresso:** Lo studente apre la pagina. L'albero è vuoto. Seleziona "Max Heap".
2. **Setup:** Preme "Start". L'algoritmo genera 15 numeri casuali. I nodi appaiono sullo schermo con un'animazione di entrata ("pop-in").
3. **Fase 1 - Heapify:**

- Lo studente esamina l'albero. Nota che il nodo 40 è figlio del nodo 20. In un Max-Heap, il padre deve essere maggiore.
- Clicca sul 40 (si illumina). Clicca sul 20.
- I nodi si scambiano di posto con un'animazione fluida. L'array sottostante si aggiorna: '[..., 40, ...]' prende il posto di '[..., 20, ...]'.
- Contemporaneamente, la CPU fa la sua mossa sulla destra.

4. Fase 2 - Extract:

- Una volta che tutto l'albero è verde (proprietà soddisfatta), lo studente clicca sulla radice (il massimo).
- La radice vola via verso l'array ordinato finale. L'ultimo nodo dell'heap prende il posto della radice (diventando temporaneamente viola per indicare la violazione).
- Il gioco riprende con la fase di Heapify sul nuovo, più piccolo, heap.

5. **Endgame:** Quando l'ultimo nodo è ordinato, compare la schermata di riepilogo: "Vittoria! Tempo: 45s - Mosse: 24".

2.3 Strumenti e Tecnologie: Analisi Tecnica

La scelta dello stack tecnologico non è stata casuale, ma guidata dal requisito di *Universalità* e *Longevità*.

2.3.1 HTML5: Semantica e Accessibilità

Non abbiamo usato 'div' generici per tutto. L'uso di tag come '<main>', '<section>', '<header>' e '<button>' garantisce che l'applicazione sia interpretabile correttamente anche dagli Screen Reader per utenti ipovedenti. Ogni nodo dell'albero ha attributi 'aria-label' e 'data-value' per esporre il proprio contenuto sia agli strumenti di assistenza che agli script di test.

2.3.2 CSS3 Avanzato: Grid, Flexbox e Variabili

Il layout dell'albero binario è una delle parti più complesse.

- **CSS Grid:** È stato utilizzato per disegnare la struttura dell'albero. Ogni livello dell'albero corrisponde a una riga della griglia; le colonne

sono calcolate per centrare perfettamente ogni nodo rispetto ai suoi figli.

- **CSS Variables (Custom Properties):** Definire colori come `--primary-color: #4CAF50` o `--node-size: 50px` in un unico punto (il blocco `:root`) ha permesso di implementare facilmente temi diversi e di mantenere la coerenza visiva.
- **Hardware Acceleration:** Per le animazioni di movimento (`transform: translate`), il browser delega il calcolo alla GPU invece che alla CPU, garantendo fluidità anche su dispositivi mobili meno potenti.

2.3.3 JavaScript (ES6+) Moderno

Il codice è scritto aderendo allo standard ECMAScript 2015+.

- **Classi e OOP:** Il paradigma a oggetti mappa perfettamente la realtà del problema. `class Node` ha proprietà `value` e `element` (riff. DOM). `class Heap` contiene la logica matematica.
- **Async/Await e Promises:** La gestione del tempo è cruciale. L'animazione dura 500ms. Il codice JavaScript non può "bloccarsi" aspettando l'animazione. Abbiamo usato le `Promise` per dire al codice: "Aspetta che la transizione CSS finisca prima di aggiornare logico i dati".
- **Modularity:** L'uso di `import/export` nativi permette di avere file piccoli e focalizzati (`Heap.js`, `Renderer.js`, `Game.js`) invece di un unico file monolitico ("Spaghetti Code"), facilitando enormemente il debugging.

2.4 Analisi Approfondita dei Requisiti

2.4.1 Requisiti Funzionali (RF)

Dettagliamo ulteriormente il comportamento atteso:

RF01 - Configurazione Partita Il sistema deve impedire l'avvio della partita se non è stata selezionata una modalità valida. Il cambio di modalità a partita in corso deve resettare lo stato corrente per evitare incoerenze (es. un Min-Heap parzialmente ordinato trattato come Max-Heap).

RF02 - Generazione Heap e Rendering L'algoritmo di generazione deve produrre numeri pseudo-casuali in un range leggibile (es. 1-100) per evitare nodi con numeri troppo lunghi che uscirebbero dai cerchi grafici. La visualizzazione deve supportare almeno 4 livelli di profondità (fino a 15 nodi).

RF03 - Interazione e Validazione Mosse Il core del gioco. Il sistema deve intercettare il click.

- *Primo Click*: Il nodo viene evidenziato (stato "Selected").
- *Secondo Click*:
 - Se è lo stesso nodo: Deseleziona.
 - Se è un nodo non adiacente (non padre/figlio): Errore visivo, reset selezione.
 - Se è adiacente ma non viola la proprietà di heap (swap inutile): Errore visivo ("Mossa non necessaria!").
 - Se è adiacente e viola la proprietà: Esegui swap, aggiorna punteggio.

RF04 - Intelligenza Artificiale (CPU) La CPU non deve "barare" risolvendo istantaneamente l'array. Deve simulare un "tempo di pensiero" e un "tempo di azione" per essere umanamente seguibile. Deve implementare una coda di azioni per non sovrapporre le animazioni.

RF05 - Rilevamento Fine Gioco e Vittoria Il sistema deve verificare la condizione di vittoria ad ogni completamento di una mossa. La vittoria è definita come: 'heapSize == 0' E 'array' è ordinato correttamente.

2.4.2 Requisiti Non Funzionali (RNF)

- **RNF01 - Performance**: Il Time-to-Interactive (TTI) all'avvio deve essere sotto 1 secondo. Le animazioni non devono scendere sotto i 30fps su mobile.
- **RNF02 - Usabilità**: Il design deve seguire il principio di "Affordance": ciò che è cliccabile deve sembrare cliccabile (cursor pointer, hover effects). I colori devono essere distinguibili anche da utenti daltonici (uso di simboli o pattern oltre al colore rosso/verde).
- **RNF03 - Portabilità**: Il layout Grid deve "collassare" in modo grazioso su tablet (Responsive Breakpoints).

- **RNF04 - Manutenibilità:** Il codice deve essere commentato seguendo lo standard JSDoc. Non devono esserci "Magic Numbers" nel codice; tutti i parametri di configurazione (velocità animazione, colori, dimensione heap) devono essere in costanti o file di configurazione.